```
$bot = new SingleQuoteBot();

Output:
Fatal error: Uncaught Error: Call to private QuoteBot::__construct() from
global scope
```

We've successfully broken our class. It can't be instantiated at all!

By changing the access level to `private`, the automatic call to `__construct()` that occurs after the statement `new QuoteBot` is evaluated isn't possible. It can only happen within the scope of a class method. This means that we can still instantiate a new `QuoteBot`, but we'll need to do it via some method that executes in the class context (object context isn't possible because we can't make `QuoteBot` objects until they've executed their `__construct` method).

Here's a class context method (note the declaration is `public static function`) that works with the `self` keyword to instantiate a new instance:

```
    public static function getInstance(): self
    {
        return new self();
    }
```

Now we can build new `SingleQuoteBot` objects like this:

```
$bot = SingleQuoteBot::getInstance();
print $bot->getQuote() . PHP_EOL;

Output:
Obviously, you're not a golfer.
```

So far, so useless. The real power of taking control of `__construct()` lies in the fact that we get to hide instance creation in the class itself. Which means we can do this:

```
class SingleQuoteBot
{
    protected static $instance = null;

    private function __construct(private string $quote)
    {
    }
```